`BookRepository` we implemented in chapter 3 to save the data. Finally, the `@EventListener(ApplicationReadyEvent.class)` annotation will trigger the test data generation after the application has completed the startup phase.

Listing 4.6 Loading book test data when the testdata profile is active

```
package com.polarbookshop.catalogservice.demo;

import com.polarbookshop.catalogservice.domain.Book;
import com.polarbookshop.catalogservice.domain.BookRepository;
import org.springframework.boot.context.event.ApplicationReadyEvent;
import org.springframework.context.annotation.Profile;
import org.springframework.context.event.EventListener;
import org.springframework.stereotype.Component;

@Component
@Profile("testdata")                                    ❶
public class BookDataLoader {
  private final BookRepository bookRepository;
  public BookDataLoader(BookRepository bookRepository) {
    this.bookRepository = bookRepository;
  }

  @EventListener(ApplicationReadyEvent.class)           ❷
  public void loadBookTestData() {
    var book1 = new Book("1234567891", "Northern Lights",
      "Lyra Silverstar", 9.90);
    var book2 = new Book("1234567892", "Polar Journey",
      "Iorek Polarson", 12.90);
    bookRepository.save(book1);
    bookRepository.save(book2);
  }
}
```

❶ Assigns the class to the testdata profile. It will be registered only when the testdata profile is active.

❷ The test data generation is triggered when an ApplicationReadyEvent is sent—that is when the application startup phase is completed.

In your development environment, you can use the `spring.profiles.active` property to set the testdata profile as active. You could set it in the application.yml file for the Catalog Service